Разработчики
Основы
Дизайн и планирование
Развивать
Google Play
сообщество
ПОИСК
язык
Студия Android
Войти

РАЗРАБОТАТЬ
Справочники
Ссылка
Образцы

фильтр
Руководства для разработчиков
Основы приложения
Устройства
Архитектура приложения
Знакомство
Руководство по архитектуре приложений
Об архитектуре приложения
Уровень пользовательского интерфейса
О слое пользовательского интерфейса
События пользовательского интерфейса
Держатели государств и государство пользовательского интерфейса
Государственное производство
Уровень домена
Уровень данных

Эта информация оказалась полезной?

События пользовательского интерфейса

События пользовательского интерфейса - это действия, которые должны обрабатываться на уровне пользовательского интерфейса, либо пользовательским интерфейсом, либо ViewModel. Наиболее распространенным типом событий являются *пользовательские события*. Пользователь создает пользовательские события, взаимодействуя с приложением, например, нажимая на экран или создавая жесты. Затем пользовательский интерфейс использует эти события с помощью обратных вызовов, таких как слушатели `onClick()`.

Ключевые термины:

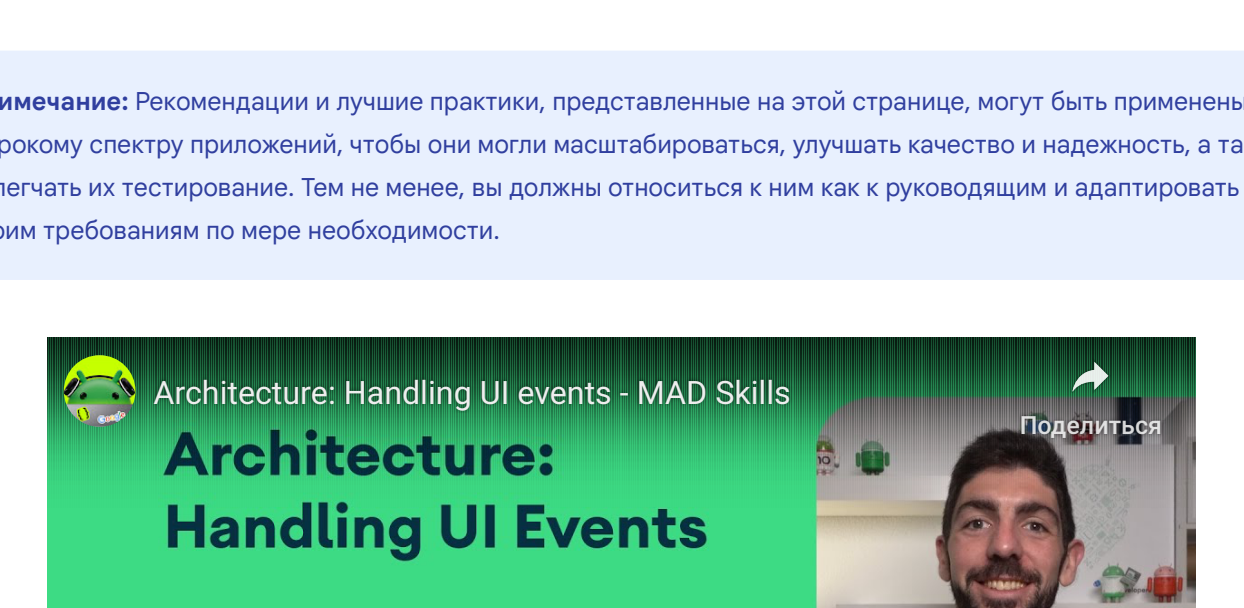
- **Пользовательский интерфейс:** код на основе представления или составления, который обрабатывает пользовательский интерфейс.
- **События пользовательского интерфейса:** Действия, которые должны обрабатываться на уровне пользовательского интерфейса.
- **События пользователя:** События, которые пользователь производит при взаимодействии с приложением.

ViewModel обычно отвечает за обработку бизнес-логики конкретного пользовательского события, например, пользователь нажимает на кнопку для обновления некоторых данных. Обычно ViewModel обрабатывает это, раскрывая функции, которые может вызвать пользовательский интерфейс. Пользовательские события также могут иметь логику поведения пользовательского интерфейса, которую пользовательский интерфейс может обрабатывать напрямую, например, переход на другой экран или отображение `Snackbar`.

В то время как *бизнес-логика* остается неизменной для одного и того же приложения на разных мобильных платформах или форм-факторах, *логика поведения пользовательского интерфейса* - это деталь реализации, которая может отличаться между этими случаями. Страница слоя пользовательского интерфейса определяет эти типы логики следующим образом:

- **Бизнес-логика** относится к *тому, что делать* с изменениями состояния, например, делать платеж или хранить предпочтения пользователя. Домен и слои данных обычно обрабатывают эту логику. В этом руководстве класс Architecture Components ViewModel используется в качестве самоуверенного решения для классов, которые обрабатывают бизнес-логику.
- **Логика поведения пользовательского интерфейса** или **логика пользовательского интерфейса** относится к *тому, как отображать* изменения состояния - например, логика навигации или как показывать сообщения пользователю. Пользовательский интерфейс обрабатывает эту логику.

Примечание: Рекомендации и лучшие практики, представленные на этой странице, могут быть применены к широкому спектру приложений, чтобы они могли масштабироваться, улучшать качество и надежность, а также облегчать их тестирование. Тем не менее, вы должны относиться к ним как к руководящим и адаптировать их к своим требованиям по мере необходимости.



Дерево решений о событии пользовательского интерфейса

На следующей диаграмме показано дерево решений, чтобы найти лучший подход к обработке конкретного случая использования события. В остальной части этого руководства подробно описаны эти подходы.

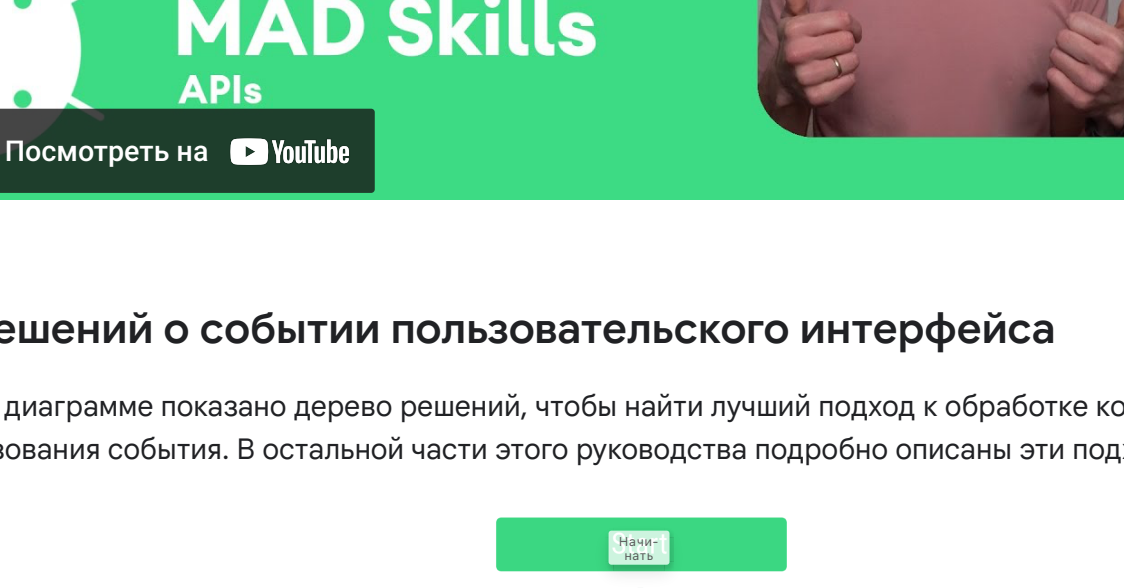

Начинать
Откуда взялось мероприятие?
UI
Просмотр модели
Когда событие требует...
Бизнес-логика
Логика поведения UI
Делегировать бизнес-логику ViewModel
Измените состояние элемента UI в UI напрямую
Обновить состояние UI

Рисунок 1. *Дерево решений для обработки событий.*

Обрабатывать пользовательские события

Пользовательский интерфейс может обрабатывать пользовательские события напрямую, если эти события связаны с изменением состояния элемента пользовательского интерфейса, например, состояния расширяемого элемента. Если событие требует выполнения бизнес-логики, такой как обновление данных на экране, оно должно быть обработано ViewModel.

Следующий пример показывает, как различные кнопки используются для расширения элемента пользовательского интерфейса (логика пользовательского интерфейса) и обновления данных на экране (бизнес-логика):

просмотры Сочинить

```
class LatestNewsActivity : AppCompatActivity() {

    private lateinit var binding: ActivityLatestNewsBinding
    private val viewModel: LatestNewsViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        /* ... */

        // The expand details event is processed by the UI that
        // modifies a View's internal state.
        binding.expandButton.setOnClickListener {
            binding.expandedSection.visibility = View.VISIBLE
        }

        // The refresh event is processed by the ViewModel that is in charge
        // of the business logic.
        binding.refreshButton.setOnClickListener {
            viewModel.refreshNews()
        }
    }
}
```

События пользователя в RecyclerViews

If the action is produced further down the UI tree, like in a `RecyclerView` item or a custom `View`, the `ViewModel` should still be the one handling user events.

For example, suppose that all news items from `NewsActivity` contain a bookmark button. The `ViewModel` needs to know the ID of the bookmarked news item. When the user bookmarks a news item, the `RecyclerView` adapter does not call the exposed `addBookmark(newsId)` function from the `ViewModel`, which would require a dependency on the `ViewModel`. Instead, the `ViewModel` exposes a state object called `NewsItemUiState` which contains the implementation for handling the event:

```
data class NewsItemUiState(
    val title: String,
    val body: String,
    val bookmarked: Boolean = false,
    val publicationDate: String,
    val onBookmark: () -> Unit
)

class LatestNewsViewModel(
    private val formatDateUseCase: FormatDateUseCase,
    private val repository: NewsRepository
)
    val newsListUiItems = repository.latestNews.map { news ->
        NewsItemUiState(
            title = news.title,
            body = news.body,
            bookmarked = news.bookmarked,
            publicationDate = formatDateUseCase(news.publicationDate),
            // Business logic is passed as a lambda function that the
            // UI calls on click events.
            onBookmark = {
                repository.addBookmark(news.id)
            }
        )
    }
}
```

This way, the `RecyclerView` adapter only works with the data that it needs: the list of `NewsItemUiState` objects. The adapter doesn't have access to the entire ViewModel, making it less likely to abuse the functionality exposed by the ViewModel. When you allow only the activity class to work with the ViewModel, you separate responsibilities. This ensures that UI-specific objects like views or `RecyclerView` adapters don't interact directly with the ViewModel.

Предупреждение: Несоверно передавать ViewModel в адаптер `RecyclerView`, потому что это тесно соединяет адаптер с классом ViewModel.

Note: Another common pattern is for the `RecyclerView` adapter to have a `Callback` interface for user actions. In that case, the activity or fragment can handle the binding and call the ViewModel functions directly from the callback interface.

Соглашения об именовании функций пользовательских событий

In this guide, the ViewModel functions that handle user events are named with a verb based on the action that they handle—for example: `addBookmark(id)` or `logIn(username, password)`.

Обрабатывать события ViewModel

Действия пользовательского интерфейса, которые происходят из событий ViewModel - ViewModel - всегда должны привести к обновлению состояния пользовательского интерфейса. Это соответствует принципам однонаправленного потока данных. Это делает события воспроизводимыми после изменений конфигурации и гарантирует, что действия пользовательского интерфейса не будут потеряны. При желании вы также можете сделать события воспроизводимыми после смерти процесса, если используете модуль сохраненного состояния.

Сопоставление действий пользовательского интерфейса с состоянием пользовательского интерфейса не всегда является простым процессом, но это приводит к более простой логике. Ваш мыслительный процесс не должен заканчиваться, например, определением того, как заставить пользовательский интерфейс перейти на определенный экран. Вам нужно подумать дальше и подумать, как представить этот пользовательский поток в вашем состоянии пользовательского интерфейса. **Другими словами: не думайте о том, какие действия должен совершить пользовательский интерфейс; подумайте о том, как эти действия влияют на состояние пользовательского интерфейса.**

Ключевой момент: события ViewModel всегда должны приводить к обновлению состояния пользовательского интерфейса.

Например, рассмотрим случай навигации на главный экран, когда пользователь вошел в систему на экране входа в систему. Вы можете смоделировать это в состоянии пользовательского интерфейса следующим образом:

```
data class LoginUiState(
    val isLoading: Boolean = false,
    val errorMessage: String? = null,
    val isUserLoggedIn: Boolean = false
)
```

This UI reacts to changes to the `isUserLoggedIn` state and navigates to the correct destination as needed:

просмотры Сочинить

```
class LoginViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(LoginUiState())
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()
    /* ... */
}

class LoginActivity : AppCompatActivity() {
    private val viewModel: LoginViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        /* ... */

        lifecycleScope.launch {
            repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.uiState.collect { uiState ->
                    if (uiState.isUserLoggedIn) {
                        // Navigate to the Home screen.
                    }
                    ...
                }
            }
        }
    }
}
```

Примечание: Примеры кода в этом разделе требуют понимания сопрограмм и того, как их использовать с компонентами, сопонимающими жизненный цикл.

Потребляющие события могут инициировать обновления состояния

Потребление определенных событий ViewModel в пользовательском интерфейсе может привести к другим обновлениям состояния пользовательского интерфейса. Например, при отображении временных сообщений на экране, чтобы пользователь знал, что что-то произошло, пользовательский интерфейс должен уведомить ViewModel, чтобы запустить еще одно обновление состояния, когда сообщение было показано на экране. Событие, которое происходит, когда пользователь потребляет сообщение (отклонив его или после тайм-аута), может рассматриваться как "пользовательский ввод", и поэтому ViewModel должен знать об этом. В этой ситуации состояние пользовательского интерфейса можно смоделировать следующим образом:

```
// Models the UI state for the Latest news screen.
data class LatestNewsUiState(
    val news: List<News> = emptyList(),
    val isLoading: Boolean = false,
    val userMessage: String? = null
)
```

ViewModel обновит состояние пользовательского интерфейса следующим образом, когда бизнес-логика требует отображения нового переходного сообщения пользователю:

просмотры Сочинить

```
class LatestNewsViewModel(/* ... */) : ViewModel() {

    private val _uiState = MutableStateFlow(LatestNewsUiState(isLoading = true))
    val uiState: StateFlow<LatestNewsUiState> = _uiState

    fun refreshNews() {
        viewModelScope.launch {
            // If there isn't internet connection, show a new message on the screen.
            if (!internetConnection()) {
                _uiState.update { currentUiState ->
                    currentUiState.copy(userMessage = "No Internet connection")
                }
                return@launch
            }

            // Do something else.
        }
    }

    fun userMessageShown() {
        _uiState.update { currentUiState ->
            currentUiState.copy(userMessage = null)
        }
    }
}
```

ViewModel не нужно знать, как пользовательский интерфейс отображает сообщение на экране; он просто знает, что есть пользовательское сообщение, которое должно быть показано. После того, как переходное сообщение было показано, пользовательский интерфейс должен уведомить ViewModel об этом, в результате чего другое обновление состояния пользовательского интерфейса очистит свойство `userMessage`:

просмотры Сочинить

```
class LatestNewsActivity : AppCompatActivity() {
    private val viewModel: LatestNewsViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        /* ... */

        lifecycleScope.launch {
            repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.uiState.collect { uiState ->
                    uiState.userMessage?.let {
                        // TODO: Show Snackbar with userMessage.

                        // Once the message is displayed and
                        // dismissed, notify the ViewModel.
                        viewModel.userMessageShown()
                    }
                    ...
                }
            }
        }
    }
}
```

Несмотря на то, что сообщение является временным, состояние пользовательского интерфейса является точным представлением того, что отображается на экране в каждый момент времени. Либо отображается сообщение пользователя, либо нет.

Примечание: Для более продвинутого сценария использования со списком пользовательских сообщений, отображаемых на экране, ознакомьтесь с образцом Jetsnack Compose.

События навигации

Потребляющие события могут инициировать обновления состояния в разделе подробностей о том, как вы используете состояние пользовательского интерфейса для отображения сообщений пользователей на экране. Навигационные события также являются распространенным типом событий в приложении для Android.

Если событие срабатывает в пользовательском интерфейсе, потому что пользователь нажал на кнопку, пользовательский интерфейс позаботится об этом, вызывая навигационный контроллер или раскрывая событие вызывающему, составящему по мере необходимости.

просмотры Сочинить

```
class LoginActivity : AppCompatActivity() {

    private lateinit var binding: ActivityLoginBinding
    private val viewModel: LoginViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        /* ... */

        binding.helpButton.setOnClickListener {
            navController.navigate(...) // Open help screen
        }
    }
}
```

Если ввод данных требует некоторой проверки бизнес-логики перед навигацией, ViewModel должен будет предоставить это состояние пользовательскому интерфейсу. Пользовательский интерфейс будет реагировать на это изменение состояния и соответствующим образом ориентироваться. Раздел событий Handle ViewModel охватывает этот пример использования. Вот похожий код:

просмотры Сочинить

```
class LoginActivity : AppCompatActivity() {
    private val viewModel: LoginViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        /* ... */

        lifecycleScope.launch {
            repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.uiState.collect { uiState ->
                    if (uiState.isUserLoggedIn) {
                        // Navigate to the Home screen.
                    }
                    ...
                }
            }
        }
    }
}
```

В приведенном выше примере приложение работает так, как ожидалось, потому что текущий пункт назначения, Login, не будет храниться в заднем стеке. Пользователи не смогут вернуться к нему, если нажмут назад. Однако в тех случаях, когда это может произойти, решение потребует дополнительной логики.

Навигационные события, когда пункт назначения хранится в заднем стеке

Когда ViewModel устанавливает некоторое состояние, которое вызывает навигационное событие с экрана A на экран B, а экран A сохраняется в обратном стеке навигации, вам может понадобиться дополнительная логика, чтобы не продолжать автоматически перемещаться на B. Для реализации этого необходимо иметь дополнительное состояние, указывающее, следует ли пользовательский интерфейс рассматривать возможность перехода на другой экран. Обычно это состояние удерживается в пользовательском интерфейсе, потому что логика навигации является проблемой пользовательского интерфейса, а не ViewModel. Чтобы проиллюстрировать это, давайте рассмотрим следующий пример использования.

Предположим, что вы находитесь в процессе регистрации вашего приложения. На экране проверки *даты рождения*, когда пользователь вводит дату, дата проверяется ViewModel, когда пользователь нажимает на кнопку "Продолжить". ViewModel делегирует логику проверки на уровень данных. Если дата действительна, пользователь переходит к следующему экрану. В качестве дополнительной функции пользователи могут перемещаться между различными экранами регистрации, если они захотят изменить некоторые данные. Таким образом, все пункты назначения в потоке регистрации хранятся в одном заднем стеке. Учитывая эти требования, вы можете реализовать этот экран следующим образом:

просмотры Сочинить

```
// Key that identifies the `validationInProgress` state in the Bundle
private const val DOB_VALIDATION_KEY = "dobValidationKey"

class DobValidationFragment : Fragment() {

    private var validationInProgress: Boolean = false
    private val viewModel: DobValidationViewModel by viewModels()

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val binding = // ...
        validationInProgress = savedInstanceState?.getBoolean(DOB_VALIDATION_KEY) ?: false

        binding.continueButton.setOnClickListener {
            viewModel.validateDob()
            validationInProgress = true
        }

        viewLifecycleOwner.lifecycleScope.launch {
            viewModel.uiState
                .flowWithLifecycle(viewLifecycleOwner.lifecycle)
                .collect { uiState ->
                    // Update other parts of the UI ...

                    // If the input is valid and the user wants
                    // to navigate, navigate to the next screen
                    // and reset `validationInProgress` flag
                    if (uiState.isDobValid && validationInProgress) {
                        validationInProgress = false
                        navController.navigate(...) // Navigate to next screen
                    }
                }
        }

        return binding
    }

    override fun onSaveInstanceState(outState: Bundle) {
        super.onSaveInstanceState(outState)
        outState.putBoolean(DOB_VALIDATION_KEY, validationInProgress)
    }
}
```

The date of birth validation is *business logic* that the ViewModel is responsible for. Most of the time, the ViewModel would delegate that logic to the data layer. The logic to navigate the user to the next screen is *UI logic* because these requirements could change depending on the UI configuration. For example, you might not want to automatically advance to another screen in a tablet if you're showing multiple registration steps at the same time. The `validationInProgress` variable in the code above implements this functionality and handles whether or not the UI should navigate automatically whenever the date of birth is valid and the user wanted to continue to the following registration step.

Другие варианты использования

Если вы считаете, что ваш случай использования события пользовательского интерфейса не может быть решен с помощью обновлений состояния пользовательского интерфейса, вам, возможно, придется пересмотреть, как данные поступают в вашем приложении. Рассмотрите следующие принципы:

- **Каждый класс должен делать то, за что он несет ответственность, а не больше.** Пользовательский интерфейс отвечает за логику поведения, специфичную для экрана, такую как навигационные вызовы, события клика и получение запросов на разрешение. ViewModel содержит бизнес-логику и преобразует результаты из нижних слоев иерархии в состояние пользовательского интерфейса.
- **Подумайте о том, где происходит мероприятие.** Следуйте дереву решений, представленному в начале этого руководства, и заставьте каждый класс заниматься тем, за что они несут ответственность. Например, если событие происходит из пользовательского интерфейса и приводит к событию навигации, то это событие должно обрабатываться в пользовательском интерфейсе. Некоторая логика может быть делегирована ViewModel, но обработка события не может быть полностью делегирована ViewModel.
- **Если у вас несколько потребителей, и вы беспокоитесь о том, что событие будет потреблено несколько раз, вам, возможно, придется пересмотреть архитектуру вашего приложения.** Наличие нескольких одновременных потребителей приводит к тому, что контракт, *поставленный ровно один раз*, становится чрезвычайно трудным для гарантии, поэтому сложность и тонкое поведение взрывается. Если у вас возникла эта проблема, подумайте о том, чтобы поднять эти проблемы вверх в дереве пользовательского интерфейса; вам может понадобиться другая сущность, о которой более высокая иерархия.
- **Подумайте о том, когда государство должно быть поглощено.** В определенных ситуациях вы можете не захотеть продолжать потреблять состояние, когда приложение находится в фоновом режиме, например, показывать тост. В этих случаях рассмотрите возможность использования состояния, когда пользовательский интерфейс находится на переднем плане.

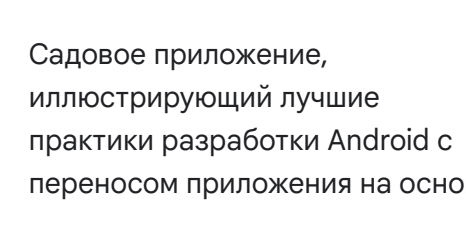
Примечание: В некоторых приложениях вы могли видеть, что события ViewModel подвергаются воздействию пользовательского интерфейса с помощью каналов Kotlin или других реактивных потоков. Когда продюсер (ViewModel) переживает потребителя (UI - Составить или Представить), эти решения не гарантируют доставку и обработку этих событий. Это может привести к будущим проблемам для разработчика, а также является неприемлемым пользовательским опытом для большинства приложений, потому что это может оставить приложение в непоследовательном состоянии, привести к ошибкам или пользователю может пропустить важную информацию.

Если вы находитесь в одной из этих ситуаций, пересмотрите, что на самом деле означает это одноразовое событие ViewModel для вашего пользовательского интерфейса. Немедленно обработайте их и сведите их до состояния пользовательского интерфейса. Состояние пользовательского интерфейса лучше представляет пользовательский интерфейс в данный момент времени, дает вам больше гарантий доставки и обработки, обычно его легче тестировать, и он последовательно интегрируется с остальной частью вашего приложения.

Чтобы узнать больше о том, почему вы не должны использовать вышеупомянутые API с некоторыми примерами кода, прочитайте пост в блоге ViewModel: One-opon event antipatterns.

Образцы

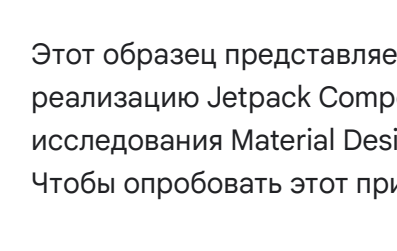
Следующие примеры Google демонстрируют события пользовательского интерфейса на уровне пользовательского интерфейса. Изучите их, чтобы увидеть это руководство на практике:



GITHUB

Подсолнух с компози

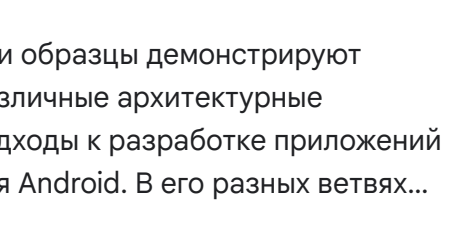
Садовое приложение, иллюстрирующий лучшие практики разработки Android с переносом приложения на основ...



GITHUB

Образец совы

Этот образец представляет собой реализацию Jetpack Compose Owl, исследования Material Design. Чтобы опробовать этот пример...



GITHUB

Архитектура

Эти образцы демонстрируют различные архитектурные подходы к разработке приложений для Android. В его разных ветвях...

Еще

Рекомендуем

Уровень пользовательского интерфейса
Откройте для себя новейшие инструменты разработки...
Обновлено 12 июля 2023 г.

Держатели государств и государство...
Откройте для себя новейшие инструменты разработки...
Обновлено 12 июля 2023 г.

Руководство по архитектуре приложений
Откройте для себя новейшие инструменты разработки...
Обновлено 12 дек. 2023 г.

Эта информация оказалась полезной?

На содержание и образцы кода на этой странице распространяются лицензии, описанные в Лицензии на контент. Java и OpenJDK являются товарными знаками или зарегистрированными товарными знаками Oracle и/или ее филиалов.

Последнее обновление 2023-07-12 UTC.

Содержание
Дерево решений о событии пользовательского интерфейса
Обрабатывать пользовательские события
События пользователя в RecyclerViews
Соглашения об именовании функций пользовательских событий
Обрабатывать события ViewModel
Потребляющие события могут инициировать обновления состояния
События навигации
Навигационные события, когда пункт назначения хранится в заднем стеке
Другие варианты использования
Образцы

Рекомендуем
Уровень пользовательского интерфейса
Обновлено 12 июля 2023 г.
Держатели государств и государство пользовательского интерфейса
Обновлено 12 июля 2023 г.
Руководство по архитектуре приложений
Обновлено 12 дек. 2023 г.

X
Подписывайтесь на @AndroidDev на X
YOUTUBE
Проверьте Android Developers на YouTube
LINKEDIN
Свяжитесь с сообществом разработчиков Android в LinkedIn

БОЛЬШЕ АНДРОИ
android
Android для предприятий
безопасность
источник
новости
блог
Подкасты

УЗНАТЬ
Игры
Машинное обучение
КОНФИДЕНЦИАЛЬНОСТЬ
5 Г

УСТРОЙСТВА ANDROID
Большие экраны
Носить ОС
Устройства ChromeOS
Android для автомобилей
Android TV

РЕЛИЗЫ
Android 14
Android 13
Андроид 12
Андроид 11
Андроид 10
Пирог
Орео
Нуга

ДОКУМЕНТАЦИЯ И ЗАГРУЗКИ
Руководство по Android Studio
Руководства для разработчиков
Ссылка на API
Скачать Studio
Android NDK

ПОДДЕРЖКА
Сообщить об ошибке платформы
Сообщить об ошибке документации
Поддержка Google Play
Присоединяйтесь к исследованиям

Google для разработчиков
android
Chrome
Огненная база
Облачная платформа Google
Все продукты

КОНФИДЕНЦИАЛЬНОСТЬ | Лицензия | Рекомендации по бренду
Получайте новости и советы по электронной почте
подписаться
язык